

XRationals.jl

Exact rational arithmetic with IEEE-like special values (NaN, Inf, -Inf), overflow-safe saturation, lazy normalization, and zero heap allocation.

Overview

XRationals.jl is built for exact rational arithmetic when you want fixed-width performance characteristics instead of arbitrary-precision growth. The package keeps numerators and denominators in compact integer fields, avoids heap allocation in normal arithmetic, and gives the exported Qx types a predictable saturation model.

The exported Qx types are the practical front door for the package: they preserve exact rational semantics while adding IEEE-like Inf, -Inf, and NaN values plus lazy normalization for faster chained operations. The public manual focuses on these five exported widths.

XRationals.jl provides five exported rational number types:

Type	Int	Overflow	Normalization
XRational32 (Qx32)	Int32	Saturates	Lazy via Int64
XRational64 (Qx64)	Int64	Saturates	Lazy via Int128
XRational128 (Qx128)	Int128	Saturates	Lazy via Int256
XRational256 (Qx256)	Int256	Saturates	Lazy via Int512
XRational512 (Qx512)	Int512	Saturates	Lazy via Int1024

Choosing a type

- **Need IEEE-like robustness and speed?** Use Qx32, Qx64, Qx128, Qx256, or Qx512. Overflow saturates to Inf/NaN, and lazy normalization gives strong speedups over Rational{Int} for chained arithmetic.
- **Do not need Inf/NaN semantics?** Use Rational{Int32}, Rational{Int64}, Rational{Int128}, Rational{Int256}, or Rational{Int512} instead.

Quick start

```
using XRationals

# Basic exact arithmetic
a = Qx32(2, 3)
b = Qx32(5, 7)
a + b  # 29//21
a * b  # 10//21
a ^ 3  # 8//27

# IEEE-like special values
Qx32(1, 0)  # Inf
```

```

Qx32(-1, 0)  # -Inf
Qx32(0, 0)   # NaN

# Overflow saturates instead of crashing
Qx32(typemax{Int32}, 1) + 1  # Inf

# Lazy normalization: GCD deferred until display
Qx64(6, 8) == Qx64(3, 4)  # true (cross-multiply comparison)

```

Key features

- **Zero heap allocation:** all arithmetic uses fixed-width integers (Int32/Int64/Int128/Int256/Int512/Int1024/Int2048)
- **Broader width coverage:** choose 32-, 64-, 128-, 256-, or 512-bit exported extended rationals from the same API shape
- **Lazy normalization:** GCD is deferred until display, hashing, or conversion
- **typemin rejection:** constructors reject typemin(Int32), typemin(Int64), typemin(Int128), typemin(Int256), and typemin(Int512) to prevent silent negation overflow
- **Fused multiply-add:** fma(x, y, z) computes $x*y + z$ with exact intermediate precision
- **Cross-width conversion:** exact widening constructors (Qx64(x::Qx32), Qx128(x::Qx32), Qx128(x::Qx64), Qx256(x::Qx32), Qx256(x::Qx64), Qx256(x::Qx128), Qx512(x::Qx32), Qx512(x::Qx64), Qx512(x::Qx128), Qx512(x::Qx256)) are the canonical widening APIs and preserve value, widen(Qx32) == Qx64, widen(Qx64) == Qx128, widen(Qx128) == Qx256, and widen(Qx256) == Qx512 expose the same widening ladder, and Qx256(x::Qx512), Qx128(x::Qx512), Qx64(x::Qx512), Qx32(x::Qx512), Qx128(x::Qx256), Qx64(x::Qx256), Qx32(x::Qx256), Qx64(x::Qx128), Qx32(x::Qx128), and Qx32(x::Qx64) compute the nearest representable narrower value
- **IEEE ordering:** NaN sorts last, Inf/-Inf compare correctly

Benchmarks

The repo includes a runnable benchmark harness at test/Benchmark.jl:

```
julia --project=. test/Benchmark.jl
```

Representative local results from that harness:

Qx32 vs Rational{Int32}

Operation	Rational{Int32}	Qx32	Speedup
a + b	13 ns	2 ns	7.0x
a * b	8 ns	2 ns	4.2x
a / b	7 ns	2 ns	3.6x
muladd(a,b,a)	25 ns	3 ns	7.5x

Operation	Rational{Int32}	Qx32	Speedup
a+b+c+d	66 ns	5 ns	14.3x
a*b-c*d	40 ns	4 ns	10.1x

Qx64 vs Rational{Int64}

Operation	Rational{Int64}	Qx64	Speedup
a + b	14 ns	3 ns	5.3x
a * b	9 ns	2 ns	4.0x
a / b	8 ns	3 ns	3.2x
muladd(a,b,a)	28 ns	6 ns	4.6x
a+b+c+d	72 ns	8 ns	9.4x
a*b-c*d	43 ns	5 ns	8.2x

Qx128 vs Rational{Int128}

Operation	Rational{Int128}	Qx128	Speedup
a + b	75 ns	7 ns	10.4x
a * b	65 ns	6 ns	10.8x
a / b	60 ns	7 ns	8.9x
muladd(a,b,a)	144 ns	12 ns	11.7x
a+b+c+d	269 ns	20 ns	13.2x
a*b-c*d	216 ns	17 ns	12.5x

Qx256 vs Rational{Int256}

Operation	Rational{Int256}	Qx256	Speedup
a + b	267 ns	23 ns	11.4x
a * b	230 ns	16 ns	14.3x
a / b	208 ns	19 ns	11.1x
muladd(a,b,a)	438 ns	38 ns	11.4x
a+b+c+d	996 ns	69 ns	14.5x
a*b-c*d	791 ns	53 ns	14.8x

Qx512 vs Rational{Int512}

Operation	Rational{Int512}	Qx512	Speedup
a + b	583 ns	93 ns	6.3x
a * b	461 ns	59 ns	7.9x
a / b	417 ns	64 ns	6.5x
muladd(a,b,a)	941 ns	156 ns	6.0x
a+b+c+d	2.1 us	310 ns	6.8x
a*b-c*d	1.7 us	222 ns	7.9x

fma is slower than stdlib Rational1 because XRationals routes finite fused multiply-add through a slower exact-or-canonical path before converting back to the fixed-width result. Use muladd when that extra guarantee is not needed.

Pages

- [Extended Rationals \(Qx32/Qx64/Qx128/Qx256/Qx512\)](#)
- [Usage Guide](#)
- [API Reference](#)